

TEMSA Digital Twin

Platform Teknik Dokümantasyon

Sistem Mimarisi · Veritabanı Tasarımı · API Referansı · Güvenlik · DevOps

Versiyon: 2.1

Tarih: 16 Nisan 2026

Hazırlayan: TEMSA Ar-Ge Dijital Mühendislik Ekibi

Gizlilik: Şirket İçi — Dağıtımı Yasaktır

Durum: Production-Ready

Katman	Teknoloji	Versiyon
Frontend	React + Vite + Tailwind CSS	18.3 / 4.x / 3.x
Backend	Python + FastAPI + Uvicorn	3.12 / 0.115 / 0.34
Veritabanı	PostgreSQL (Async) + Redis	16-alpine / 7-alpine
ORM	SQLAlchemy (AsyncSession)	2.0+
Auth	JWT (HS256) + bcrypt + OAuth2	python-jose / bcrypt 4.2
DevOps	Docker Compose (8 konteyner)	Multi-service
Monitoring	Health checks + Import logs	Otomatik

İÇİNDEKİLER

1. Yönetici Özeti
2. Sistem Mimarisi Genel Bakış
3. Docker Altyapısı ve Konteyner Mimarisi
4. Veritabanı Tasarımı (PostgreSQL)
5. Backend API Mimarisi (FastAPI)
6. API Endpoint Referansı
7. Kimlik Doğrulama ve Yetkilendirme
8. Frontend Mimarisi (React)
9. Veri Akışı ve İş Mantığı
10. BOM Entegrasyon Sistemi
11. Performans, Ölçeklenebilirlik ve Cache
12. Güvenlik Mimarisi
13. Migration ve Veritabanı Evrim Stratejisi
14. Test ve Kalite Güvence
15. Deployment ve DevOps Süreci
16. Sonuç ve Teknik Değerlendirme

1. Yönetici Özeti

TEMSA Digital Twin platformu, TEMSA otobüs araçlarının Avrupa Birliği CO₂ regülasyonları (EU 2019/1242 ve EU 2019/631) kapsamında zorunlu olan VECTO (Vehicle Energy Consumption Calculation Tool) simülasyon verilerini merkezi bir sistemde yönetmek, analiz etmek ve raporlamak için tasarlanmış **kurumsal düzeyde bir dijital ikiz platformudur**.

Platform; araç varyant yönetimi, motor/şanzıman/lastik konfigürasyonlarının detaylı kaydı, yakıt haritası ve yük eğrisi verilerinin saklanması, CO₂ emisyon hesaplamaları, filo bazlı ağırlıklı ortalama hesaplamaları, gerçek test sonuçlarıyla korelasyon analizi ve BOM (Bill of Materials) entegrasyon süreçlerini tek bir çatı altında birleştirir.

Temel Metrikler

Metrik	Değer	Açıklama
Toplam API Endpoint	78+	10 router modülü üzerinden sunulan RESTful endpoint
Veritabanı Tablosu	17+	Ana DB (11 tablo + 3 view) + BOM DB (6+ tablo)
Frontend Sayfa/Panel	19	React SPA — 7 navigasyon grubu
Docker Konteyner	8	PostgreSQL x2, Redis, FastAPI x3, React, Flask
Veritabanı İndeksi	14+	Performans-optimize edilmiş B-tree indeksleri
Migration Sayısı	6	Artımlı şema evrimi — v1'den v6'ya
Kullanıcı Rolü	7	admin, manager, analyst, viewer, engineering, entegration, design

Bu dokümantasyon, platformun her katmanını — veritabanı şemasından API endpoint imzalarına, güvenlik mekanizmalarından deployment pipeline'ına kadar — bir senior mühendis perspektifinden detaylı şekilde ele almaktadır.

2. Sistem Mimarisini Genel Bakış

Platform, modern mikroservis-benzeri (micro-service-like) bir mimari üzerine inşa edilmiştir. Her servis kendi Docker konteynerinde izole çalışır, servisler arası iletişim Docker internal network üzerinden gerçekleşir. Bu yaklaşım, bağımsız ölçeklendirme, hata izolasyonu ve bağımsız deployment imkânı sağlar.

2.1 Mimari Katmanlar

Sistem dört ana katmandan oluşur: **Sunum Katmanı** (React SPA), **API Katmanı** (FastAPI + Flask), **Veri Katmanı** (PostgreSQL + Redis) ve **Orkestrasyon Katmanı** (Docker Compose).

Katman	Teknoloji	Sorumluluk	Port
Sunum	React 18.3 + Vite	SPA UI, routing, responsive tasarım	3000
API Gateway	Vite Dev Proxy	CORS, path-based routing	—
Ana Backend	FastAPI 0.115	REST API, async I/O, validasyon	8000
BOM Servisi	FastAPI (ayrı)	Excel entegrasyonu, malzeme, takvim	8001
Simülasyon	FastAPI (Simutem)	EV enerji simülasyonu	8002
Homologasyon	Flask 3.x	Regülasyon scraping, checklist	5001
Veri Katmanı	PostgreSQL 16	ACID ilişkisel veri depolama	5433/5434
Cache	Redis 7	Session cache, rate limiting	6379

2.2 Servisler Arası İletişim

Frontend, farklı backend servislerine Vite proxy konfigürasyonu üzerinden erişir. **/api/v1/*** istekleri ana backend'e (port 8000), **/bom-api/*** istekleri BOM backend'e (port 8001), **/simutem-api/*** istekleri simülasyon servisine (port 8002), **/homolog-api/*** istekleri homologasyon servisine (port 5001) yönlendirilir.

- Frontend → Backend: HTTP/REST (JSON), Vite reverse proxy ile
- Backend → PostgreSQL: asyncpg bağlantı havuzu (pool_size=20, max_overflow=10)
- Backend → Redis: aioredis async client
- Servisler arası: Docker internal network (bridge mode)

2.3 Veri Akış Diyagramı (Mantıksal)

Kaynak	İşlem	Hedef	Format
VECTO XML Dosyası	XML parse → model eşleştirme	vehicle_variants	XML → JSON → SQL
VECTO Sonuç XML	CO ₂ /FC extraction	vecto_results_certified	XML → Numeric
Excel BOM Dosyası	Satır parse → seviye atama	bom_items	XLSX → JSON → SQL
Kullanıcı Girişi	JWT token üretimi	Redis + users tablosu	Form → bcrypt → JWT
RSLT_MANUFACTURER	Çoklu misyon ayrıştırma	vecto_simulation_outputs	XML → Multi-row

3. Docker Altyapısı ve Konteyner Mimarisi

Tüm servisler Docker Compose ile orkestre edilir. Tek bir **docker-compose up -d** komutu ile 8 konteyner ayağa kalkar. Servisler arası bağımlılıklar `depends_on` + `healthcheck` mekanizması ile yönetilir, böylece veritabanı hazır olmadan backend başlamaz.

3.1 Konteyner Envanteri

Konteyner	Image	Port	Volume	Healthcheck
temsa-db	postgres:16-alpine	5433:5432	pgdata + init.sql	pg_isready
temsa-redis	redis:7-alpine	6379:6379	redisdata	—
temsa-backend	python:3.12 (custom)	8000:8000	app, vecto_files, outputs	—
temsa-frontend	node:18 (custom)	3000:3000	src, public	—
temsa-bom-db	postgres:16-alpine	5434:5432	bom_pgdata	pg_isready
temsa-bom-backend	python:3.12 (custom)	8001:8000	uploads	—
temsa-simutem	python:3.12 (custom)	8002:8000	—	—
temsa-homolog	python:3.x (custom)	5001:5000	—	—

3.2 Volume Yönetimi

Docker named volume'lar sayesinde konteyner yeniden oluşturulsa bile veri kaybolmaz. Veritabanı verileri **pgdata** ve **bom_pgdata** volume'larında, Redis verileri **redisdata** volume'unda kalıcı olarak tutulur.

Volume Adı	Mount Hedefi	Kalıcılık	İçerik
pgdata	/var/lib/postgresql/data	Evet	Ana veritabanı (temsa_twin)
bom_pgdata	/var/lib/postgresql/data	Evet	BOM veritabanı (bomdb)
redisdata	/data	Evet	Redis AOF/RDB snapshot
bom_uploads	/app/uploads	Evet	Yüklenen Excel dosyaları
./vecto_files	/app/vecto_files (bind)	Host	VECTO XML kaynak dosyaları
./vecto_outputs	/app/vecto_outputs (bind)	Host	VECTO simülasyon çıktıları
./public	/app/public (bind)	Host	Statik varlıklar (fontlar, görseller)

3.3 Network Topolojisi

Docker Compose varsayılan olarak bir bridge network oluşturur. Tüm konteynerler bu network üzerinde birbirine hostname ile erişir (örn. backend konteyneri veritabanına **db:5432** adresi ile bağlanır). Dış dünyaya sadece port mapping ile tanımlanan portlar açıktır.

- İç ağ: temsa-digital-twin_default (bridge)
- DNS çözümü: Docker embedded DNS (konteyner adı → IP)
- Dış erişim: localhost:3000 (UI), localhost:8000 (API), localhost:5433 (DB admin)

4. Veritabanı Tasarımı (PostgreSQL)

Sistem iki ayrı PostgreSQL 16 instance kullanır: **temsa_twin** (ana veritabanı — araç, varyant, simülasyon, CO₂, kullanıcı verileri) ve **bomdb** (BOM entegrasyon veritabanı — proje, malzeme, takvim verileri). Her iki veritabanı UUID primary key stratejisi kullanır ve uuid-oss extension'ı ile otomatik UUID üretimi desteklenir.

4.1 PostgreSQL Konfigürasyonu

Parametre	temsa_twin	bomdb
Image	postgres:16-alpine	postgres:16-alpine
Kullanıcı	temsa	bomuser
Dış Port	5433	5434
Extension	uuid-oss, pg_trgm	uuid-oss
Encoding	UTF-8	UTF-8
Connection Pool	pool_size=20, max_overflow=10	pool_size=20
Healthcheck	pg_isready (5s interval)	pg_isready (5s interval)

4.2 ENUM Tipleri

Veritabanı, tekrar eden kategorik değerler için PostgreSQL native ENUM tiplerini kullanır. Bu, veri tutarlılığını uygulama seviyesinde değil veritabanı seviyesinde garanti eder.

ENUM Adı	Değerler	Kullanıldığı Tablo
vehicle_category	coach, city, ev, diesel	vehicles.category
engine_type	diesel, electric, hybrid	vehicle_variants.engine_type
simulation_source	vecto, matlab, custom	simulation_runs.source
simulation_status	pending, processing, completed, failed	simulation_runs.status
test_type	track, road, endurance, customer	real_test_results.test_type

4.3 Ana Tablo Şeması — vehicles

Araç model bilgilerini tutan üst düzey tablo. Her araç birden fazla varyanta sahip olabilir (1:N).

Kolon	Tip	Kısıtlama	Açıklama
id	UUID	PK, DEFAULT uuid_generate_v4()	Benzersiz araç tanımlayıcı
model_name	VARCHAR(100)	NOT NULL	Araç model adı (örn: LD SB)
manufacturer	VARCHAR(200)	NOT NULL, DEFAULT 'TEMSA'	Üretici firma adı
category	vehicle_category	NOT NULL	coach / city / ev / diesel
subcategory	VARCHAR(50)	NULLABLE	Alt kategori
legislative_category	VARCHAR(10)	DEFAULT 'M3'	AB tip onay kategorisi
chassis_config	VARCHAR(50)	NULLABLE	Şasi konfigürasyonu
axle_config	VARCHAR(20)	NULLABLE	Aks konfigürasyonu (4x2, 6x2)

Kolon	Tip	Kısıtlama	Açıklama
base_config	JSON	DEFAULT '{}'	Temel konfigürasyon
created_at	TIMESTAMPTZ	DEFAULT NOW()	Kayıt oluşturma zamanı
updated_at	TIMESTAMPTZ	ON UPDATE trigger	Son güncelleme zamanı

4.4 Ana Tablo Şeması — vehicle_variants

Sistemin en kritik ve en geniş tablosu. Her VECTO XML dosyasından parse edilen tüm teknik detaylar bu tabloda saklanır. 60'tan fazla kolon ile motor, şanzıman, lastik, aerodinamik, ADAS ve HVAC bilgilerini kapsar.

Kolon	Tip	Açıklama
id	UUID PK	Benzersiz tanımlayıcı
vehicle_id	UUID FK	Üst araç ref (CASCADE)
variant_code	VARCHAR(50)	VECTO varyant kodu
filename	VARCHAR(255)	XML dosya adı
engine_type	ENUM	diesel / electric / hybrid
engine_manufacturer	VARCHAR(200)	Motor üreticisi
engine_model	VARCHAR(200)	Motor modeli
engine_cert_number	VARCHAR(200)	AB sertifika no
displacement_cc	INTEGER	Motor hacmi (cm ³)
rated_speed_rpm	INTEGER	Nominal devir
rated_power_w	INTEGER	Nominal güç (W)
max_torque_nm	NUMERIC(8,2)	Maks tork (Nm)
idling_speed_rpm	INTEGER	Rölanti devri
fuel_type	VARCHAR(50)	Yakıt tipi
max_laden_mass_kg	INTEGER	Azami kütle (kg)
curb_weight_kg	INTEGER	Boş ağırlık (kg)
aero_cd_a	NUMERIC(6,4)	CdA (m ²)
gearbox_*	VARCHAR(200)	Üretici, model, tip (3 kolon)
gear_count	INTEGER	Vites sayısı
axle_ratio / type	NUMERIC / VARCHAR	Diferansiyel (2 kolon)
tyre_*	VARCHAR / NUMERIC	Lastik detayı (12 kolon)
ADAS kolonları	BOOLEAN / VARCHAR	stop_start, eco_roll, cruise
fan / steering / alt	VARCHAR	Yardımcı donanım (3 kolon)
pneumatic_config	JSON	Pnömatik konfigürasyon
hvac_config	JSON	Klima/ısıtma konfig
whtc_*	NUMERIC(8,5)	WHTC düzeltme (3 kolon)
bf_cold_hot	NUMERIC(8,5)	Yakıt düzeltmesi
cf_reg_per / cf_ncv	NUMERIC(8,5)	NCV düzeltmeleri

Kolon	Tip	Açıklama
fleet_count	INTEGER	Filo adedi
zero_emission	BOOLEAN	Sıfır emisyon
raw_xml_data	JSON	Ham XML verisi
vecto_schema_ver	VARCHAR(20)	VECTO versiyonu
import_date	TIMESTAMPTZ	Aktarma tarihi
is_active	BOOLEAN	Soft delete
created/updated_at	TIMESTAMPTZ	Zaman damgaları

4.5 Mühendislik Veri Tabloları

VECTO simülasyonu için gereken detaylı mühendislik verileri ayrı tablolarda normalize edilmiş şekilde saklanır. Her tablo vehicle_variants tablosuna FK ile bağlıdır ve CASCADE delete kuralı uygulanır.

Tablo	Kolonlar	Amaç	Veri Örneği
fuel_consumption_maps	speed, torque, fc	Yakıt haritası	2100rpm, 800Nm → 42.5g/s
full_load_drag_curves	speed, max_torque, drag	Yük/sürtünme eğrileri	1800rpm → 1100/-85 Nm
gear_ratios	gear_number, ratio	Vites oranları	Vites 6 → 0.78
torque_converter_chars	speed_r, torque_r, input	Tork konvertör	0.8 → 1.2 ratio
axle_loss_maps	speed, torque, loss	Aks kayıp haritası	500rpm, 2000Nm → 35.5

4.6 Simülasyon ve Test Sonuç Tabloları

Tablo	Kayıt Tipi	Anahtar Metrikler
simulation_runs	Simülasyon çalıştırma	source, status, params
simulation_results	Sonuç	co2, fuel, energy metrikleri
real_test_results	Test sonucu	test tipi, koşullar
vecto_results_certified	VECTO sertifika (56 kol.)	VIN, misyon×yükleme, CO ₂ /FC
vecto_simulation_outputs	VECTO çıktı	cf_actual, WHTC faktörleri
import_logs	İçe aktarma	dosya, status, hata, sayılar

4.7 vecto_results_certified — Detaylı Şema

Bu tablo, AB regülasyonu gereği resmi VECTO sertifikalı sonuçları saklar. **56 kolon** ile sistemin en kapsamlı tablosudur. Her VIN için birden fazla misyon×yükleme kombinasyonu kaydedilir (örn: Urban-LowLoad, Suburban-RefLoad, Interurban-FullLoad).

Kolon Grubu	Kolonlar	Açıklama
Kimlik	id, variant_id, vin, vehicle_model	Araç tanımlama
Sınıflandırma	vehicle_category, vehicle_group, group_co2, class_bus, axle_config	AB sınıflandırma
Kütle	corrected_actual_mass, tech_max_laden_mass, total_vehicle_mass, payload, mass_passengers	Ağırlık (5 kolon, _kg)
Misyon	mission, loading, primary_subgroup, distance_km, passenger_count	Simülasyon senaryoları
Hız	avg_speed, avg_driving_speed, max_speed	Hız profili (_kmh)
CO ₂	co2_g_per_km, co2_g_per_pkm	g/km ve g/yolcu-km
Yakıt	fc_g_per_km, fc_mj_per_km, fc_l_per_100km + 3 pkm	6 birimde yakıt tüketimi
Enerji	energy_mj_per_km	Enerji (MJ/km)
Verimlilik	gearbox_eff, axlegear_eff, gearshift_count	Aktarma verimliliği
Motor	rated_power, capacity, fuel_type, propulsion_power	Motor parametreleri

Kolon Grubu	Kolonlar	Açıklama
Şanzıman	transmission_type, nr_of_gears, retarder, axle_ratio	Şanzıman detayları
Lastik	average_rrc, tyre_dimension	Lastik parametreleri
ADAS	stop_start, eco_roll, predictive_cruise	Sürüş destek
HVAC	hvac_config, double_glazing	İklimlendirme
Özet	is_summary, summary_co2, summary_fc	Ağırlıklı ort.
Meta	source_file, source_type, tool_version, sim_date, status	Veri kaynağı

4.8 Kullanıcı ve Rol Tabloları

RBAC (Role-Based Access Control) sistemi iki tablo ile yönetilir. Roller, JSON formatında izin dizileri (permissions) taşır ve her kullanıcı tek bir role atanır.

Rol	İzinler (JSONB)	Açıklama
admin	["*"]	Tam yetki
manager	["view", "edit", "import", "export"]	Yönetici — veri girişi ve analiz
analyst	["view", "analyze", "export"]	Analist — salt okunur + analiz
viewer	["view"]	İzleyici — salt okunur
engineering	["weightcalc", "simutem"]	Mühendislik modülleri
entegration	["bom", "materials"]	Entegrasyon — BOM
design	["view", "export", "compare"]	Tasarım

4.9 İndeks Stratejisi

Sık sorgulanan kolonlar üzerinde B-tree indeksler tanımlanmıştır. pg_trgm extension'ı sayesinde trigram bazlı benzerlik aramaları da desteklenir.

İndeks	Tablo.Kolon	Tip
idx_vehicles_category	vehicles.category	B-tree
idx_vehicles_model	vehicles.model_name	B-tree
idx_variants_vehicle	vehicle_variants.vehicle_id	B-tree (FK)
idx_variants_code	vehicle_variants.variant_code	B-tree (UNIQUE)
idx_variants_engine	vehicle_variants.engine_type	B-tree
idx_variants_filename	vehicle_variants.filename	B-tree
idx_fcm_variant	fuel_consumption_maps.variant_id	B-tree (FK)
idx_fldc_variant	full_load_drag_curves.variant_id	B-tree (FK)
idx_vrc_vin	vecto_results_certified.vin	B-tree
idx_vrc_variant	vecto_results_certified.variant_id	B-tree (FK)
idx_vrc_summary	vecto_results_certified.is_summary	B-tree
idx_vrc_subgroup	vecto_results_certified.primary_subgroup	B-tree

İndeks	Tablo.Kolon	Tip
idx_simruns_variant	simulation_runs.variant_id	B-tree (FK)
idx_simruns_status	simulation_runs.status	B-tree
idx_importlogs_filename	import_logs.filename	B-tree

4.10 View — v_variant_summary

Sık kullanılan varyant listesi sorgusu için materialized olmayan bir view tanımlanmıştır. Bu view, vehicles ve vehicle_variants tablolarını JOIN eder ve alt sorgu ile yakıt haritası nokta sayısı (fcm_points), yük eğrisi nokta sayısı (fldc_points) ve gerçek vites sayısı (gear_count_actual) bilgilerini ekler.

4.11 Trigger — updated_at Otomatik Güncelleme

update_updated_at_column() fonksiyonu, vehicles ve vehicle_variants tablolarında her UPDATE işleminde updated_at kolonunu otomatik olarak NOW() değerine ayarlar. Bu, uygulama katmanında unutulabilecek zaman damgası güncellemesini veritabanı seviyesinde garanti eder.

5. Backend API Mimarisi (FastAPI)

Backend, Python 3.12 üzerinde FastAPI framework ile geliştirilmiştir. Tamamen **asenkron** (async/await) I/O modeli kullanır. Uvicorn ASGI sunucusu üzerinde çalışır. SQLAlchemy 2.0+ AsyncSession ile veritabanı işlemleri non-blocking gerçekleşir.

5.1 Proje Yapısı

Dosya/Dizin	Sorumluluk
main.py	FastAPI app oluşturma, CORS, router kayıt, lifespan yönetimi
config.py	pydantic-settings ile ortam değişkenleri (.env desteği)
database.py	SQLAlchemy async engine, session factory, Base class, get_db dependency
models.py	SQLAlchemy ORM modelleri (17+ model)
schemas.py	Pydantic v2 şemaları — request/response validasyonu
routers/vehicles.py	Araç ve varyant CRUD — 9 endpoint
routers/co2_v2.py	CO ₂ , filo hesaplama, digital twin — 25 endpoint
routers/auth.py	Kimlik doğrulama, kullanıcı CRUD — 9 endpoint
routers/analysis.py	Analiz, sıralama, karşılaştırma — 4 endpoint
routers/simulations.py	XML import, bulk upload — 5 endpoint
routers/vsum_analytics.py	VSUM derin analitik — 11 endpoint
routers/pdf_report.py	PDF rapor üretimi — 2 endpoint
routers/dashboard.py	Hava durumu, haberler — 2 endpoint
utils/xml_parser.py	VECTO XML parse fonksiyonları

5.2 Konfigürasyon Yönetimi

Tüm konfigürasyon pydantic-settings ile yönetilir. .env dosyasından otomatik okuma yapılır. Hassas bilgiler (SECRET_KEY, DB_PASSWORD) production'da ortam değişkeni olarak set edilir.

Parametre	Tip	Varsayılan	Açıklama
APP_NAME	str	TEMSA Digital Twin	Uygulama adı
APP_VERSION	str	1.0.0	Semantik versiyon
DEBUG	bool	True	Geliştirme modu (SQL echo)
DATABASE_URL	str	postgresql+asyncpg://...	Async DB bağlantısı
DATABASE_URL_SYNC	str	postgresql://...	Senkron bağlantı (migration)
REDIS_URL	str	redis://redis:6379/0	Redis bağlantısı
VECTO_IMPORT_DIR	str	/app/vecto_files	VECTO dosya dizini
SECRET_KEY	str	(değiştirilecek)	JWT imzalama anahtarı
JWT_ALGORITHM	str	HS256	JWT algoritması
ACCESS_TOKEN_EXPIRE_MINUTES	int	480 (8 saat)	JWT token ömrü

5.3 Veritabanı Bağlantı Havuzu

SQLAlchemy async engine, asyncpg driver ile bağlantı havuzu yönetir. **pool_size=20** ile eşzamanlı 20 bağlantı, **max_overflow=10** ile ani yük durumunda ek 10 bağlantı açılabilir. Toplam maksimum: 30 eşzamanlı veritabanı bağlantısı.

- engine = create_async_engine(DATABASE_URL, pool_size=20, max_overflow=10, echo=DEBUG)
- AsyncSession: expire_on_commit=False — lazy load sorunu önlenir
- Dependency Injection: get_db() async generator — otomatik session yaşam döngüsü

5.4 CORS Politikası

- allow_origins: ['http://localhost:3000', 'http://localhost:5173']
- allow_credentials: True (cookie/JWT taşınabilir)
- allow_methods: ['*'] — tüm HTTP metotları
- allow_headers: ['*'] — tüm headerlar

6. API Endpoint Referansı

Sistemde toplam **78+ RESTful endpoint** bulunmaktadır. Tüm endpointler JSON formatında veri alışverişi yapar (application/json), dosya yüklemeleri multipart/form-data kullanır.

6.1 Araç ve Varyant API (9 Endpoint)

Prefix: /api/v1 | **Tags:** Vehicles & Variants

Metot	Path	Response	Açıklama
GET	/vehicles	list[VehicleResponse]	Tüm araçlar. ?category= filtresi
GET	/vehicles/{id}	Vehicle + variants	Araç detayı + varyant listesi
POST	/vehicles	VehicleResponse	Yeni araç oluştur
GET	/variants	list[VariantResponse]	Varyant listesi. ?vehicle_id= ?search=
GET	/variants/{id}	VariantDetail	Varyant tam detayı (60+ alan)
GET	/variants/{id}/fuel-map	list[FuelMapPoint]	Motor yakıt haritası noktaları
GET	/variants/{id}/load-curves	list[LoadCurvePoint]	Tam yük/sürtünme eğrisi
GET	/variants/{id}/gear-ratios	list[GearRatioItem]	Vites oranları listesi
POST	/variants/compare	list[ComparisonItem]	2-10 varyant karşılaştırma

6.2 CO₂ ve Filo API (25 Endpoint)

Prefix: /api/v1/co2 | **Tags:** CO₂ & Fleet

Metot	Path	Açıklama
POST	/import-result	Tek VECTO sonuç XML yükle (RSLT_CUSTOMER / RSLT_MANUFACTURER)
POST	/import-result-directory	Dizinden toplu sonuç import
GET	/fleet-emissions	Filo bazlı CO ₂ emisyon özeti
GET	/digital-twin/{variant_code}	Varyant dijital ikiz detayı
GET	/digital-twin-list	Tüm dijital ikiz listesi
POST	/migrate-tyres	Lastik verisi migration (v3→v4)
GET	/variants-hub	Varyant hub — özet metrikler
POST	/compare-models	Model bazlı karşılaştırma
POST	/compare-variants	Varyant detaylı karşılaştırma
POST	/import-output-directory	VECTO çıktı dizini import
GET	/variant-results	VIN bazlı sonuç listesi
GET	/variant-results/{vin}	VIN misyon sonuçları
GET	/fleet-co2-calculation	Filo ağırlıklı ortalama CO ₂
POST	/fleet-count	Filo araç sayısı güncelle
GET	/fleet-tracking	Filo takip verileri
POST	/test-data	Gerçek test sonucu ekle

Metot	Path	Açıklama
GET	/correlation	Simülasyon ↔ gerçek test korelasyonu
GET	/fleets	Filo listesi
POST	/fleets	Yeni filo oluştur
GET	/fleets/{id}	Filo detayı + üyeleri
PUT	/fleets/{id}	Filo güncelle
DELETE	/fleets/{id}	Filo sil
POST	/fleets/compare	Filolar arası karşılaştırma
GET	/benchmark	Sektör kıyaslama verileri
GET	/vecto-results	Tüm sertifikalı VECTO sonuçları

6.3 Kimlik Doğrulama API (9 Endpoint)

Prefix: /api/v1/auth | Tags: auth

Metot	Path	Yetki	Açıklama
POST	/login	—	Kullanıcı adı/şifre ile JWT token al
GET	/me	Bearer	Mevcut kullanıcı profili
POST	/change-password	Bearer	Şifre değiştir (eski doğrulamalı)
GET	/users	Admin	Tüm kullanıcıları listele (+roller)
POST	/users	Admin	Yeni kullanıcı oluştur
PUT	/users/{id}	Admin	Kullanıcı güncelle
DELETE	/users/{id}	Admin	Kullanıcı sil (kendini silemez)
POST	/users/{id}/reset-password	Admin	Şifreyi varsayılabilecek şekilde sıfırla
GET	/roles	Bearer	Tüm rolleri listele

6.4 Analiz API (4 Endpoint)

Prefix: /api/v1/analysis | Tags: Analysis & Insights

Metot	Path	Açıklama
GET	/rankings	Metrik bazlı varyant sıralaması (?metric= ?category=)
GET	/insights	Otomatik üretilen iç görüler ve tavsiyeler
GET	/compare-detailed	Çoklu varyant detaylı karşılaştırma
GET	/fleet-summary	Filo özet istatistikleri

6.5 Diğer API Grupları

Grup (Endpoint Sayısı)	Prefix	Temel İşlevler
VSUM Analytics (11)	/api/v1/vsum-analytics	Enerji Sankey, vites dağılımı, yükleme hassasiyeti, ADAS etkisi
İçe Aktarma (5)	/api/v1 (import/...)	Tekli/toplu XML yükleme, izin import, log takibi, istatistikler
PDF Rapor (2)	/api/pdf-report	PDF rapor üretimi ve önizleme

Grup (Endpoint Sayısı)	Prefix	Temel İşlevler
Dashboard (2)	/api/v1/dashboard	Hava durumu ve haber akışı
BOM API (35+)	/bom-api (ayrı servis)	Proje CRUD, malzeme, takvim, takip, maliyet, audit, entegrasyon

7. Kimlik Doğrulama ve Yetkilendirme

Sistem, endüstri standardı JWT (JSON Web Token) tabanlı kimlik doğrulama kullanır. Kullanıcı şifreleri bcrypt ile hash'lenir (salt round otomatik), tokenlar HS256 algoritması ile imzalanır.

7.1 Kimlik Doğrulama Akışı

Adım	İşlem	Detay
1	POST /api/v1/auth/login	Kullanıcı adı + şifre gönderilir
2	bcrypt.checkpw()	Şifre, veritabanındaki hash ile karşılaştırılır
3	JWT token üretimi	sub=username, role=role_name, exp=now+480min
4	Token döndürülür	{access_token, token_type, user: {id, username, role, ...}}
5	Her istek: Authorization	Bearer <token> formatında gönderilir
6	get_current_user()	Token decode → users JOIN roles → kullanıcı dict
7	Rol kontrolü	require_admin() veya endpoint bazlı kontrol

7.2 JWT Token Yapısı

Header: {"alg": "HS256", "typ": "JWT"}

Payload: {"sub": "username", "role": "admin", "exp": 1718928000}

Signature: HMACSHA256(header + payload, SECRET_KEY)

- Token ömrü: 480 dakika (8 saat) — bir iş günü boyunca geçerli
- Token yenileme: Mevcut implementasyonda refresh token yok, expiry'de yeniden login
- OAuth2PasswordBearer: FastAPI dependency injection ile otomatik token yakalama

7.3 Şifre Güvenliği

- Hash algoritması: bcrypt (adaptive cost function)
- Salt: bcrypt.gensalt() — her hash'te benzersiz salt
- Doğrulama: bcrypt.checkpw(password.encode(), hashed) — timing-safe karşılaştırma
- Varsayılan admin şifresi: Seed data ile oluşturulur, production'da değiştirilmeli
- Şifre sıfırlama: Admin tarafından, varsayılan şifreye döner

7.4 Yetkilendirme Matrisi

İşlem	admin	manager	analyst	viewer	engineering	entegration
Veri görüntüleme						
Veri import			—	—	—	—
Veri export				—	—	
Analiz çalıştırma				—	—	—

İşlem	admin	manager	analyst	viewer	engineering	entegration
Kullanıcı yönetimi		—	—	—	—	—
BOM işlemleri			—	—	—	
Ağırlık hesaplam a		—	—	—		—
Homologasyon		—	—	—		—

8. Frontend Mimarisi (React)

Frontend, React 18.3 ile geliştirilmiş tek sayfa uygulamasıdır (SPA). Vite 4.x build tool, Tailwind CSS 3.x utility-first CSS framework, Motion (Framer Motion) 11.18 animasyon kütüphanesi ve Recharts 2.15 grafik kütüphanesi kullanılır.

8.1 Navigasyon Yapısı (7 Grup, 19 Sayfa)

Grup	Sayfa	Bileşen	İşlev
Genel	dashboard	DashboardPage	Gösterge paneli — istatistikler, grafikler
	variants	VariantList / VariantDetail	Araç varyant listesi ve detay
	variant-outputs	VariantOutputsPanel	VECTO çıktı dosyaları yönetimi
Mühendislik	co2	CO2Panel	CO ₂ emisyon analizi ve raporlama
	fleet-calculation	FleetCalculationPanel	Filo bazlı CO ₂ hesaplama
	digital-twin	DigitalTwinPanel	Araç dijital ikiz görselleştirme
	weight	WeightCalcPanel	Ağırlık hesaplama modülü
	enerji	EnerjiAnaliziPanel	VSUM enerji analizi
	benchmark	BenchmarkPanel	Sektör kıyaslama
	bom	BomProjectsPanel / Detail	BOM proje yönetimi (çift görünüm)
Entegrasyon	materials	MaterialsPanel	Malzeme veritabanı
	import	ImportPanel	VECTO XML içe aktarma
Simülasyon	virtual-test	VirtualTestPanel	Sanal test senaryoları
Filo & Analiz	fleet-tracking	FleetTrackingPanel	Filo gerçek zamanlı takip
	insights	InsightsPanel	Otomatik analiz bulguları
	rankings	RankingsPanel	Varyant performans sıralaması
Homologasyon	homologasyon	HomologasyonPanel	Regülasyon takip sistemi
Yönetim	admin	AdminPanel	Kullanıcı ve rol yönetimi (admin only)

8.2 Teknoloji Stack Detayı

Teknoloji	Versiyon	Kullanım Alanı
React	18.3	UI bileşen kütüphanesi — hooks, context, SPA
Vite	4.x	Build tool — HMR, proxy, ESBuild bundling
Tailwind CSS	3.x	Utility-first CSS — responsive, dark mode
Motion (Framer)	11.18.0	Sayfa geçişleri, bileşen animasyonları
Recharts	2.15	Grafik bileşenleri (Area, Bar, Radar, Pie)
jsPDF	—	İstemci-tarafli PDF üretimi
html2canvas	—	DOM → canvas → PDF dönüşümü
lucide-react	—	SVG ikon kütüphanesi (500+ ikon)

8.3 State Management ve Context

- **AuthContext:** Kullanıcı oturum durumu, JWT token, login/logout fonksiyonları
- **ThemeContext:** Tema yönetimi (dark/light mode geçişi)
- **Yerel state:** useState/useEffect ile bileşen bazlı veri yönetimi
- **API katmanı:** api.js (ana backend) + bomApi.js (BOM servisi) — merkezi fetch wrapper

8.4 API İstemci Katmanı

Frontend'de iki API istemci modülü bulunur. **api.js** ana backend ile, **bomApi.js** BOM servisi ile iletişimi yönetir. Her iki modül de fetch API üzerinde merkezi hata yönetimi, JWT token ekleme ve base URL konfigürasyonu sağlar.

Modül	Base URL	Fonksiyon Sayısı	Kategoriler
api.js	/api/v1	40+	Dashboard, Vehicles, Variants, Analysis, Import, CO ₂ , Digital Twin, Fleet, VSUM
bomApi.js	/bom-api	35+	Projects, Materials, Calendar, SubTasks, FollowUps, Costs, Audit, Integration

9. Veri Akışı ve İş Mantığı

9.1 VECTO XML İçe Aktarma Süreci

VECTO XML dosyaları iki farklı endpoint üzerinden sisteme aktarılır. Tekli yükleme (/import/upload) ve toplu yükleme (/import/upload-bulk). Her iki durumda da aynı parse pipeline çalışır:

Adım	İşlem	Detay
1	Dosya alımı	multipart/form-data ile XML dosya(lar) alınır
2	XML Parse	ElementTree ile XML ağacı oluşturulur
3	Model eşleştirme	XML'deki araç adı ile vehicles tablosu sorgulanır; yoksa oluşturulur
4	Varyant oluşturma	60+ alan XML'den extract edilir → vehicle_variants INSERT
5	Alt veri extract	Yakıt haritası, yük eğrisi, vites oranları → bulk INSERT
6	Tork konvertör	Varsa tork konvertör karakteristiği extract + INSERT
7	Aks kayıp haritası	Aks kayıp verileri extract + INSERT
8	İmport log	import_logs tablosuna başarı/hata kaydı + kayıt sayıları

9.2 CO₂ Sonuç Import Süreci

VECTO simülasyon sonuç dosyaları (RSLT_CUSTOMER, RSLT_MANUFACTURER) ayrı bir pipeline ile işlenir. Bu dosyalar birden fazla misyon×yükleme kombinasyonu içerir.

- Dosya tipi tespiti: XML root element analizi (RSLT_CUSTOMER vs RSLT_MANUFACTURER)
- VIN eşleştirme: Sonuç dosyasındaki VIN ile vehicle_variants tablosu sorgulanır
- Çoklu kayıt: Her misyon×yükleme kombinasyonu ayrı bir satır olarak kaydedilir
- Özet satırı: is_summary=True ile ağırlıklı ortalama değerler ayrıca saklanır

9.3 Filo CO₂ Hesaplama Mantığı

AB regülasyonu gereği filo bazlı ağırlıklı ortalama CO₂ değeri hesaplanır. Her varyantın fleet_count değeri ağırlık katsayısı olarak kullanılır.

Formül: $\text{Filo CO}_2 = \frac{\sum(\text{varyant_co2} \times \text{fleet_count})}{\sum(\text{fleet_count})}$

- Misyon bazlı hesaplama: Urban, Suburban, Interurban ayrımı
- Yükleme bazlı hesaplama: LowLoad, RefLoad, FullLoad ayrımı
- Birim: g/km, g/pkm, l/100km — üç farklı birimde raporlama

9.4 Digital Twin Veri Akışı

Digital Twin modülü, bir varyantın tüm teknik verilerini tek bir görünümde birleştirir: araç konfigürasyonu, motor parametreleri, yakıt haritası 3D yüzeyi, yük eğrileri, vites oranları, simülasyon sonuçları ve gerçek test karşılaştırması.

- Endpoint: GET /api/v1/co2/digital-twin/{variant_code}
- Veri kaynakları: vehicle_variants + fuel_consumption_maps + full_load_drag_curves + gear_ratios + vecto_results_certified + real_test_results
- Response: Tek JSON objesi içinde tüm ilişkili veriler (nested)

9.5 Korelasyon Analizi

Simülasyon sonuçları (vecto_results_certified) ile gerçek test sonuçları (real_test_results) arasındaki sapma analiz edilir. Bu, VECTO simülasyonunun güvenilirliğini doğrulamak için kritik bir metriktir.

- Eşleştirme: variant_id üzerinden JOIN
- Sapma hesabı: $(\text{gerçek} - \text{simülasyon}) / \text{simülasyon} \times 100 = \% \text{ sapma}$
- Görselleştirme: Scatter plot (simülasyon vs gerçek) + regression line

10. BOM Entegrasyon Sistemi

BOM (Bill of Materials) entegrasyon sistemi, ayrı bir FastAPI servisi olarak çalışır (port 8001) ve kendi PostgreSQL veritabanına (bomdb, port 5434) sahiptir. Excel dosyalarından BOM verisi import eder, seviye hiyerarşisi oluşturur ve SAP entegrasyonu için veri hazırlar.

10.1 BOM Servisi Mimarisi

Bileşen	Teknoloji	Sorumluluk
bom-backend	FastAPI + SQLAlchemy	REST API, Excel parse, iş mantığı
bom-db	PostgreSQL 16	Proje, malzeme, takvim, maliyet verileri
Frontend (paylaşımlı)	React (BomProjectsPanel)	Proje listesi, detay, entegrasyon UI

10.2 BOM API Kategorileri (35+ Endpoint)

Kategori	Endpoint Sayısı	Temel İşlevler
Proje Yönetimi	12	Upload, CRUD, navigation, export, kalem tipi, reprocess
Malzeme Veritabanı	6	CRUD, MM03 import, arama, sayaç
Takvim (Test)	8	Otobüs CRUD, görev CRUD, bottleneck, Excel/PDF export
Alt Görevler	4	Görev altı iş kalemleri CRUD
Takip (Follow-Up)	4	Durum takibi — proje bazlı filtreleme
Maliyet	5	Maliyet kaydı CRUD + özet raporu
Denetim (Audit)	2	İşlem log kaydı + geçmiş sorgu
Entegrasyon	9	SAP entegrasyon upload, onay süreci, şablon

11. Performans, Ölçeklenebilirlik ve Cache

11.1 Async I/O Mimarisi

FastAPI'nin async/await desteği sayesinde tüm veritabanı ve I/O işlemleri non-blocking gerçekleşir. Bu, tek bir Python process'in yüzlerce eşzamanlı isteği event loop ile yönetmesine olanak tanır — thread-per-request modeline göre çok daha verimli.

- **asyncpg:** PostgreSQL C-extension driver — sync psycopg2'ye göre 2-3x hızlı
- **AsyncSession:** Sorgu sırasında event loop bloklanmaz
- **pool_size=20 + max_overflow=10:** Maksimum 30 eşzamanlı DB bağlantısı

11.2 Redis Cache Stratejisi

Redis 7, session cache, sık sorgulanan verilerin cache'lenmesi ve rate limiting için kullanılır. Kalıcı veri depolama (AOF/RDB) aktiftir.

- **Dashboard istatistikleri:** 5 dakika TTL ile cache'lenir
- **Varyant listesi:** Kategori bazlı cache key ile saklanır
- **Analiz sonuçları:** Parameter hash → result cache

11.3 Veritabanı Optimizasyonu

- **14+ B-tree indeks:** Sık sorgulanan kolonlar (category, variant_code, vin, status)
- **pg_trgm extension:** Trigram bazlı benzerlik araması (%LIKE% sorgularını hızlandırır)
- **JSON kolonlar:** Esnek şema gerektiren veriler (pneumatic_config, hvac_config, raw_xml_data)
- **Partial index potansiyeli:** is_active=True, is_summary=True gibi filtreler
- **View:** v_variant_summary — sık kullanılan JOIN'i önceden tanımlar

11.4 Ölçeklenebilirlik Yolu

Mevcut mimari dikey ölçeklendirme (vertical scaling) için optimize edilmiş olup, yatay ölçeklendirme (horizontal scaling) için aşağıdaki stratejiler uygulanabilir:

- **Backend replika:** Docker Compose'da backend servisini scale=3 ile çoğaltma
- **PostgreSQL read replica:** Analiz sorgularını replica'ya yönlendirme
- **Redis Cluster:** Cache kapasitesini artırma
- **CDN:** Frontend static asset'leri CDN üzerinden dağıtma

12. Güvenlik Mimarisi

Platform, OWASP Top 10 güvenlik risklerine karşı çoklu savunma katmanı uygular.

Katman	Mekanizma	Korunan Risk
Kimlik Doğrulama	JWT + bcrypt + OAuth2	Yetkisiz erişim
Yetkilendirme	RBAC (7 rol, permission dizileri)	Yetki yükseltme
Input Validasyon	Pydantic v2 şema validasyonu	Injection saldırıları
SQL Injection	SQLAlchemy ORM (parameterized queries)	Veritabanı sızıntısı
CORS	Whitelist bazlı origin kontrolü	Cross-origin saldırıları
XSS	React DOM escaping + JSON API	Script injection
Ortam İzolasyonu	Docker konteyner izolasyonu	Sistem erişimi
Secret Yönetimi	.env dosyası + ortam değişkenleri	Credential sızıntısı
Veri Bütünlüğü	FK constraints + CASCADE + NOT NULL	Tutarsız veri
Audit Trail	import_logs + audit_logs tabloları	İzlenebilirlik

12.1 OWASP Uyumluluk Detayı

OWASP Risk	Uygulanan Önlem
A01: Broken Access Control	RBAC + JWT middleware + admin-only endpointler
A02: Cryptographic Failures	bcrypt hash + HS256 JWT + production'da secret rotation
A03: Injection	SQLAlchemy parameterized queries + Pydantic validasyon
A04: Insecure Design	Async architecture + proper error handling + input bounds
A05: Security Misconfiguration	Docker izolasyon + .env bazlı konfigürasyon
A06: Vulnerable Components	Pinned versions (requirements.txt) + Alpine base images
A07: Auth Failures	bcrypt + timing-safe comparison + account lockout (roadmap)
A08: Software/Data Integrity	Docker image integrity + volume persistence + FK constraints
A09: Logging Failures	import_logs + audit_logs + structured logging
A10: SSRF	Backend-only external calls + URL whitelist

13. Migration ve Veritabanı Evrim Stratejisi

Veritabanı şeması, artımlı (incremental) migration dosyaları ile evrimleştirilir. Her migration dosyası geri dönüşümsüz (forward-only) ALTER TABLE ve CREATE TABLE ifadeleri içerir. Bu yaklaşım, production veritabanında veri kaybı olmadan şema değişikliği yapılmasını sağlar.

Migration	İçerik	Eklenen Kolon/Tablo
init.sql (v1)	Temel şema oluşturma	11 tablo + 14 indeks + trigger + view + 5 ENUM
migration_v2.sql	CO ₂ sertifika desteği	fleet_count + vecto_results_certified (56 kolon) + 3 indeks
migration_v3.sql	Detaylı simülasyon metrikleri	9 kolon (primary_subgroup, distance, payload, hız, verimlilik) + 1 indeks
migration_v4.sql	Ön/arka lastik ayrımı	12 kolon (front/rear × manufacturer, model, dim, rrc, fz_iso, twin_tyres)
migration_v5.sql	Kullanıcı yönetimi	roles + users tablosu + 4 seed rol + admin kullanıcı + 2 indeks
migration_v6.sql	Departman bazlı roller	3 yeni rol: engineering, entegration, design (JSONB permissions)

13.1 Migration Uygulama Sırası

Migration dosyaları sıralı olarak çalıştırılmalıdır. init.sql Docker container ilk oluşturulduğunda otomatik çalışır (/docker-entrypoint-initdb.d/). Sonraki migration'lar manuel veya CI/CD pipeline üzerinden uygulanır.

- init.sql → migration_v2.sql → migration_v3.sql → migration_v4.sql → migration_v5.sql → migration_v6.sql
- Her migration idempotent tasarlanmıştır (IF NOT EXISTS kontrolleri)
- Geri alma (rollback): Her migration için ayrı rollback SQL dosyası hazırlanmalıdır (roadmap)

14. Test ve Kalite Güvence

14.1 Test Stratejisi

Katman	Araç	Kapsam
Birim Test (Unit)	pytest + pytest-asyncio	Model validasyonu, XML parser, hesaplama
Entegrasyon Test	TestClient (httpx)	API endpoint'leri, DB işlemleri, auth akışı
E2E Test	Tarayıcı bazlı (potansiyel)	Kullanıcı senaryoları, iş akışı doğrulaması
Veri Doğrulama	Pydantic v2 şemaları	Her request/response otomatik validasyon
Tip Güvenliği	Python type hints + mypy	Statik tip analizi

14.2 Veri Kalite Kontrolleri

- XML parse başarısızlığı: import_logs'a hata mesajı ile kayıt
- Duplicate kontrolü: variant_code UNIQUE constraint — tekrar import engellenir
- Referential integrity: FK constraints ile; parent silinirse CASCADE ile alt veriler temizlenir
- NOT NULL kısıtlamaları: Kritik alanlar (model_name, variant_code, vin) boş bırakılamaz
- JSON şema validasyonu: Pydantic v2 ile nested JSON yapıları doğrulanır

15. Deployment ve DevOps Süreci

15.1 Deployment Komutu

Tüm sistem tek bir komut ile başlatılır:

```
docker-compose up -d --build
```

15.2 Deployment Akışı

Adım	Komut/İşlem	Açıklama
1	git pull origin main	Son kod değişikliklerini çek
2	docker-compose build	Değişen imajları yeniden oluştur
3	docker-compose up -d	Konteynerleri arka planda başlat
4	docker-compose logs -f backend	Backend loglarını izle
5	Sağlık kontrolü	GET /health → {status: 'ok'} doğrula
6	Migration (gerekirse)	psql ile migration SQL dosyasını çalıştır
7	Smoke test	Frontend erişimi + API endpoint testi

15.3 Ortam Değişkenleri

Production deployment'ta aşağıdaki ortam değişkenleri .env dosyası veya Docker environment bölümünde set edilmelidir:

Değişken	Development	Production
DB_PASSWORD	temsa_secure_2024	Güçlü rastgele şifre
BOM_DB_PASSWORD	bompass123	Güçlü rastgele şifre
SECRET_KEY	temsa-digital-twin-secret...	Kriptografik rastgele key
DEBUG	True	False
CORS origins	localhost:3000, :5173	Production domain

15.4 Health Check ve Monitoring

- **GET /:** → {app, version, docs: '/docs', status: 'running'}
- **GET /health:** → {status: 'ok'} — load balancer probe endpointi
- **PostgreSQL:** pg_isready healthcheck (5s interval, 5 retry)
- **İmport logları:** Her dosya import işlemi import_logs tablosunda kaydedilir
- **Swagger UI:** /docs adresinde interaktif API dokümantasyonu (auto-generated)

16. Sonuç ve Teknik Değerlendirme

TEMSA Digital Twin platformu, otomotiv endüstrisinin CO₂ regülasyon gereksinimlerini karşılamak üzere tasarlanmış, **production-grade bir kurumsal yazılım sistemidir**. Aşağıdaki tabloda platformun teknik olgunluk değerlendirmesi özetlenmektedir.

Kriter	Değerlendirme	Detay
Mimari Olgunluk		Mikroservis-benzeri, Docker izolasyonu, async I/O
Veritabanı Tasarımı		Normalize şema, 17+ tablo, ENUM, FK cascade, trigger, view, 14+ indeks
API Tasarımı		78+ RESTful endpoint, OpenAPI/Swagger, Pydantic validasyon
Güvenlik		JWT + RBAC + bcrypt + CORS + SQL injection koruması
Ölçeklenebilirlik		Async I/O + connection pool hazır, horizontal scaling roadmap'te
Test Kapsamı		Pydantic validasyon aktif, birim test kapsamı genişletilebilir
Dökümantasyon		Bu belge + Swagger UI + inline kod yorumları
DevOps		Docker Compose, healthcheck, volume persistence
Kullanıcı Deneyimi		19 sayfa, animasyonlu UI, responsive tasarım, dark mode
İş Değeri		AB CO ₂ regülasyonu uyumu, filo yönetimi, BOM entegrasyon

Bu platform; veri modelleme, API tasarımı, güvenlik, frontend deneyimi ve deployment süreçleri açısından kurumsal yazılım geliştirme standartlarını karşılamaktadır. Tek bir geliştirme ekibi tarafından, modern teknoloji stack'i ile inşa edilmiş bu sistem, TEMSA'nın dijital dönüşüm yolculuğunun temel taşıdır.

Toplam Kod Satırı (Backend)	15.000+ (Python)
Toplam Kod Satırı (Frontend)	25.000+ (React/JSX)
Toplam API Endpoint	78+ (10 router modülü)
Veritabanı Tablosu	17+ (2 PostgreSQL instance)
Docker Konteyner	8 (4 backend + 2 DB + 1 cache + 1 frontend)
Frontend Sayfa/Panel	19 bileşen
Kullanıcı Rolü	7 (RBAC)
Migration Dosyası	6 (artımlı şema evrimi)
Varyant Kolon Sayısı	60+ (vehicle_variants)
Sertifika Sonuç Kolonu	56 (vecto_results_certified)